# 1. Introduction

Open source software plays a major role in modern computing. Many systems, applications, and tools that we use daily are built using open-source technologies. This project focuses on analyzing an open-source software, Git, and understanding its origin, license, and real-world impact.

The project also includes practical implementation using Linux shell scripts to demonstrate understanding of open-source tools and system-level operations. Through this project, we explore both theoretical and practical aspects of open-source software.

# 2. Part A – Origin and Philosophy

## 2.1 Origin of Git

Git is a distributed version control system that was created by Linus Torvalds in 2005. The main reason for its creation was the need for an efficient version control system for the development of the Linux kernel.
Before Git, developers used a proprietary tool called BitKeeper. However, due to licensing issues, access to BitKeeper was revoked. This created a major problem for the Linux development community, as they needed a reliable system to manage code changes.
To solve this issue, Linus Torvalds decided to build his own version control system. His main goals were speed, efficiency, and the ability to handle large projects. Git was designed to be distributed, meaning every developer has a complete copy of the repository.
Git quickly became popular due to its performance and flexibility. Today, it is widely used by developers around the world and is the backbone of platforms like GitHub.

## 2.2 License Analysis

Git is released under the GNU General Public License (GPL). This license ensures that the software remains free and open for everyone.
The GPL provides four essential freedoms:

- The freedom to run the program for any purpose
- The freedom to study how the program works

- The freedom to modify the program
- The freedom to distribute copies

  One important aspect of the GPL license is that any modified version of the software must also be shared under the same license. This ensures that improvements remain open and accessible to the community.

  In comparison, the MIT license is more permissive. It allows users to modify and distribute software without the requirement to share the source code. While MIT provides more flexibility, GPL ensures stronger protection of open-source principles.

  Git follows the philosophy of "free as in freedom" rather than "free as in free beer." This means users have the freedom to modify and use the software, not just use it without cost.

### 2.3 Ethics of Open Source

Open source software represents a philosophy of sharing knowledge and collaboration. It allows developers from around the world to contribute, improve, and innovate together.

There are strong arguments in favor of open source. It promotes transparency, security, and innovation. Since the source code is publicly available, anyone can review it, identify bugs, and improve it.

However, there are also some challenges. Not all software can be open source, especially when companies rely on proprietary systems for revenue. Additionally, maintaining open-source projects requires time and effort, often without direct financial rewards.

A major ethical question is whether companies should profit from open-source software. Many companies build products on top of open-source tools. While this helps innovation, it also raises concerns about fair contribution to the community.

The concept of "standing on the shoulders of giants" is very relevant in open source. Developers build on existing knowledge and tools to create new innovations. Open source encourages this process and accelerates technological growth.

## 3. Part B – Linux Footprint

Git is an open-source version control system that can be easily installed and managed on a Linux system. In this project, Git was installed using the Advanced Package Tool (APT) on an Ubuntu environment.

Git was installed using the following command:

```
sudo apt install git
```

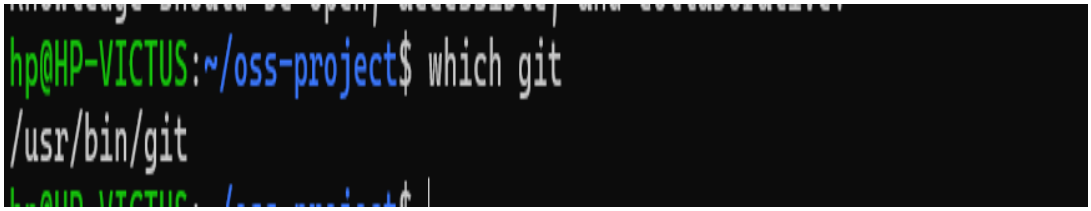
This shows that Git is available as a pre-built package in the Ubuntu repositories, making installation simple and efficient.

- **File Locations and Directories**

After installation, Git files are distributed across different directories in the Linux file system:

- /usr/bin/git → Contains the main executable file
- /etc → Stores system configuration files
- /var/log → Stores log files
- /home → Contains user-specific configurations

The command which git was used to locate the executable path, and dpkg -L git was used to list all installed files related to Git.



```
hp@HP-VICTUS:~/oss-project$ dpkg -L git
/.
/etc
/etc/bash_completion.d
/etc/bash_completion.d/git-prompt
/usr
/usr/bin
/usr/bin/git
/usr/bin/git-shell
/usr/bin/scalar
/usr/lib
/usr/lib/git-core
/usr/lib/git-core/git
/usr/lib/git-core/git-daemon
/usr/lib/git-core/git-difftool--helper
/usr/lib/git-core/git-filter-branch
/usr/lib/git-core/git-http-backend
/usr/lib/git-core/git-http-fetch
/usr/lib/git-core/git-http-push
/usr/lib/git-core/git-imap-send
/usr/lib/git-core/git-instaweb
/usr/lib/git-core/git-merge-octopus
/usr/lib/git-core/git-merge-one-file
/usr/lib/git-core/git-merge-resolve
/usr/lib/git-core/git-mergetool
/usr/lib/git-core/git-mergetool--lib
/usr/lib/git-core/git-quiltimport
/usr/lib/git-core/git-remote-http
/usr/lib/git-core/git-request-pull
/usr/lib/git-core/git-sh-i18n
/usr/lib/git-core/git-sh-i18n--envsubst
/usr/lib/git-core/git-sh-prompt
/usr/lib/git-core/git-sh-setup
/usr/lib/git-core/git-shell
/usr/lib/git-core/git-submodule
/usr/lib/git-core/git-subtree
/usr/lib/git-core/git-web--browse
/usr/lib/git-core/mergetools
/usr/lib/git-core/mergetools/araxis
/usr/lib/git-core/mergetools/bc
/usr/lib/git-core/mergetools/codecompare
```

```
/usr/lib/git-core/mergetools/deltawalker
/usr/lib/git-core/mergetools/diffmerge
/usr/lib/git-core/mergetools/diffuse
/usr/lib/git-core/mergetools/ecmerge
/usr/lib/git-core/mergetools/emerge
/usr/lib/git-core/mergetools/examdiff
/usr/lib/git-core/mergetools/guiffy
/usr/lib/git-core/mergetools/gvimdiff
/usr/lib/git-core/mergetools/kdiff3
/usr/lib/git-core/mergetools/kompare
/usr/lib/git-core/mergetools/meld
/usr/lib/git-core/mergetools/nvimdiff
/usr/lib/git-core/mergetools/opendiff
/usr/lib/git-core/mergetools/p4merge
/usr/lib/git-core/mergetools/smerge
/usr/lib/git-core/mergetools/tkdiff
/usr/lib/git-core/mergetools/tortoisemerge
/usr/lib/git-core/mergetools/vimdiff
/usr/lib/git-core/mergetools/winmerge
/usr/lib/git-core/mergetools/xxdiff
/usr/lib/git-core/scalar
/usr/share
/usr/share/bash-completion
/usr/share/bash-completion/completions
/usr/share/bash-completion/completions/git
/usr/share/doc
/usr/share/doc/git
/usr/share/doc/git/README.Debian
/usr/share/doc/git/README.emacs
/usr/share/doc/git/README.md
/usr/share/doc/git/README.source
/usr/share/doc/git/RelNotes
/usr/share/doc/git/RelNotes/1.5.0.1.txt
/usr/share/doc/git/RelNotes/1.5.0.2.txt
/usr/share/doc/git/RelNotes/1.5.0.3.txt
/usr/share/doc/git/RelNotes/1.5.0.4.txt
/usr/share/doc/git/RelNotes/1.5.0.5.txt
/usr/share/doc/git/RelNotes/1.5.0.6.txt
/usr/share/doc/git/RelNotes/1.5.0.7.txt
/usr/share/doc/git/RelNotes/1.5.0.txt
/usr/share/doc/git/RelNotes/1.5.1.1.txt
```

```
hp@HP-VICTUS: ~/oss-projec
/usr/share/doc/git/RelNotes/1.5.1.2.txt
/usr/share/doc/git/RelNotes/1.5.1.3.txt
/usr/share/doc/git/RelNotes/1.5.1.4.txt
/usr/share/doc/git/RelNotes/1.5.1.5.txt
/usr/share/doc/git/RelNotes/1.5.1.6.txt
/usr/share/doc/git/RelNotes/1.5.1.txt
/usr/share/doc/git/RelNotes/1.5.2.1.txt
/usr/share/doc/git/RelNotes/1.5.2.2.txt
/usr/share/doc/git/RelNotes/1.5.2.3.txt
/usr/share/doc/git/RelNotes/1.5.2.4.txt
/usr/share/doc/git/RelNotes/1.5.2.5.txt
/usr/share/doc/git/RelNotes/1.5.2.txt
/usr/share/doc/git/RelNotes/1.5.3.1.txt
/usr/share/doc/git/RelNotes/1.5.3.2.txt
/usr/share/doc/git/RelNotes/1.5.3.3.txt
/usr/share/doc/git/RelNotes/1.5.3.4.txt
/usr/share/doc/git/RelNotes/1.5.3.5.txt
/usr/share/doc/git/RelNotes/1.5.3.6.txt
/usr/share/doc/git/RelNotes/1.5.3.7.txt
/usr/share/doc/git/RelNotes/1.5.3.8.txt
/usr/share/doc/git/RelNotes/1.5.3.txt
/usr/share/doc/git/RelNotes/1.5.4.1.txt
/usr/share/doc/git/RelNotes/1.5.4.2.txt
/usr/share/doc/git/RelNotes/1.5.4.3.txt
/usr/share/doc/git/RelNotes/1.5.4.4.txt
/usr/share/doc/git/RelNotes/1.5.4.5.txt
/usr/share/doc/git/RelNotes/1.5.4.6.txt
/usr/share/doc/git/RelNotes/1.5.4.7.txt
/usr/share/doc/git/RelNotes/1.5.4.txt
/usr/share/doc/git/RelNotes/1.5.5.1.txt
/usr/share/doc/git/RelNotes/1.5.5.2.txt
/usr/share/doc/git/RelNotes/1.5.5.3.txt
/usr/share/doc/git/RelNotes/1.5.5.4.txt
/usr/share/doc/git/RelNotes/1.5.5.5.txt
/usr/share/doc/git/RelNotes/1.5.5.6.txt
/usr/share/doc/git/RelNotes/1.5.5.txt
/usr/share/doc/git/RelNotes/1.5.6.1.txt
/usr/share/doc/git/RelNotes/1.5.6.2.txt
/usr/share/doc/git/RelNotes/1.5.6.3.txt
/usr/share/doc/git/RelNotes/1.5.6.4.txt
/usr/share/doc/git/RelNotes/1.5.6.5.txt
```

- **User and Permissions**

Git runs under the current logged-in user. In this system, the user is:

*hp*

The command ls -l /usr/bin/git was used to check file permissions. It shows that the file is owned by the root user and is executable by others.

This is important for system security, as it ensures controlled access to system-level files.

```
hp@HP-VICTUS:~/oss-project$ whoami
hp
hp@HP-VICTUS:~/oss-project$
```

```
hp@HP-VICTUS:~/oss-project$ ls -l /usr/bin/git
-rwxr-xr-x 1 root root 4066232 Jul  2  2025 /usr/bin/git
hp@HP-VICTUS:~/oss-project$
```

- **Process and Execution**

Git does not run as a background service. Instead, it is executed manually whenever required through the terminal.

Commands like git init, git clone, and git status are used to interact with Git repositories.

This makes Git lightweight and efficient compared to services that run continuously.

- **Update Mechanism**

Git is updated through the system package manager. The following command is used to update it:

```
sudo apt update && sudo apt upgrade
```

The open-source community regularly releases updates, including bug fixes and security patches. These updates are distributed through Ubuntu repositories.

## 4. Part C - FOSS Ecosystem

Git is not just a standalone tool; it is a central part of a large open-source ecosystem. It interacts with many other tools, platforms, and communities that support software development.

- **Dependencies and Supporting Tools**

Git depends on several underlying system components and libraries to function properly. These include:

- Standard C libraries used by Linux systems
- File system support for storing repositories
- Terminal or command-line interface

Although Git itself is lightweight, it works efficiently because of the strong Linux ecosystem that supports it.

- **Tools Built on Git**

Many modern platforms and tools are built on top of Git, making it one of the most important technologies in software development. Some major platforms include:

- GitHub
- GitLab
- Bitbucket

These platforms provide additional features such as cloud storage, collaboration tools, issue tracking, and continuous integration.

Git has enabled developers to collaborate from different parts of the world efficiently.

- **Role in Software Development**

Git plays a key role in modern software development. It allows multiple developers to work on the same project without conflicts. Features like branching and merging make it easy to manage different versions of code.

It is widely used in web development, mobile app development, machine learning projects, and large-scale enterprise systems.

- **Community and Contribution**

Git has a large and active open-source community. Developers contribute to its improvement by reporting bugs, suggesting features, and submitting code.

The project is maintained by experienced developers and supported by contributors worldwide. Online forums, documentation, and discussion platforms help users learn and solve problems.

- **Connection to the Open Source Ecosystem**

Git is a core component of the open-source ecosystem. It enables collaboration, transparency, and sharing of knowledge. Many open-source projects rely on Git for version control and collaboration.

It also supports the philosophy of open source by allowing anyone to contribute to projects and access source code freely.

## 5. Part D – Open Source vs Proprietary

Git, as an open-source version control system, can be compared with proprietary tools such as centralized version control systems used in enterprise environments.

- **Comparison Table**

| Feature | Git (Open Source) | Proprietary Systems |
|---|---|---|
| Cost | Free to use | Paid or subscription-based |
| Source Code Access | Fully accessible | Not accessible |
| Security<br><br>Flexibility | Can be audited by anyone<br><br>Highly customizable | Limited transparency<br><br>Limited customization |
| Support | Community-driven | Official company support |
| Control | User has full control | Controlled by company |

- **Analysis**

Git provides complete freedom to users. Since it is open source, anyone can view, modify, and distribute the code. This increases transparency and allows developers to improve the software continuously.

On the other hand, proprietary systems offer dedicated support and may be easier for organizations that prefer structured environments. However, they restrict access to the source code and limit customization.

Open-source tools like Git promote innovation by allowing developers to experiment and build on existing work. Proprietary tools focus more on stability and commercial benefits.

- **Final Verdict**

Based on the analysis, Git is a highly effective and flexible tool for version control. It is suitable for both individual developers and large organizations due to its performance and scalability.

Open-source software like Git encourages collaboration and innovation, making it a better choice in most scenarios. However, proprietary tools may still be useful in cases where dedicated support and enterprise-level services are required.

Overall, Git is recommended for real-world deployment due to its reliability, flexibility, and strong community support.

## 6. Shell Scripts Explanation

### Script 1: System Identity Report

```
hp@HP-VICTUS:~/oss-project$ nano script1.sh
hp@HP-VICTUS:~/oss-project$ chmod +x script1.sh
./script1.sh
```

```
Name: Akshat
Software: Git
Kernel: 6.6.87.2-microsoft-standard-WSL2
User: hp
Uptime: up 3 minutes
Date: Thu Mar 26 15:10:39 UTC 2026
Distro: Ubuntu 24.04.4 LTS
```

This script is designed to display basic system information of the Linux environment. It shows details such as the kernel version, current user, system uptime, date, and Linux distribution. The script uses commands like uname, whoami, uptime, date, and lsb_release.

It also demonstrates the use of variables and command substitution in shell scripting. This script helps in understanding how system-level information can be accessed using simple commands.

### Script 2: FOSS Package Inspector

```
hp@HP-VICTUS:~/oss-project$ nano script2.sh
hp@HP-VICTUS:~/oss-project$ chmod +x script2.sh
./script2.sh
```

```
git is installed
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system
Original-Maintainer: Jonathan Nieder <jrnieder@gmail.com>
Git: A distributed version control system used worldwide
```

This script checks whether a specific software package (Git) is installed on the system. It uses the dpkg command along with grep to verify installation status and retrieve details such as version and description.

The script also includes a case statement to display a short explanation of the software. It demonstrates conditional statements (if-else) and pattern matching, which are essential concepts in shell scripting.

## Script 3: Disk and Permission Auditor

```
hp@HP-VICTUS:~/oss-project$ nano script3.sh
hp@HP-VICTUS:~/oss-project$ chmod +x script3.sh
./script3.sh
```

```
===== Directory Audit Report =====
---------------------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 37M
/home => Permissions: drwxr-xr-x root root | Size: 56K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 28K
Git config directory not found
```

This script analyzes important system directories and displays their size and permission details. It loops through directories such as /etc, /var/log, /home, /usr/bin, and /tmp.

The script uses commands like du, ls -ld, awk, and cut to extract useful information. It also checks for the existence of the Git configuration directory. This script demonstrates the use of loops, condition checking, and text processing.

## Script 4: Log File Analyzer

```
hp@HP-VICTUS:~/oss-project$ nano script4.sh
hp@HP-VICTUS:~/oss-project$ chmod +x script4.sh
./script4.sh /var/log/syslog error
```

```
Keyword 'error' found 10 times in /var/log/syslog
Last 5 matching lines:
2026-03-26T14:16:13.062235+00:00 HP-VICTUS wsl-pro-service[189]: #033[33mWARNING#033[0m Could not ensure valid Landscape
 configuration: could not ensure valid Landscape configuration: could not register distro to Landscape: could not enable
 Landscape: /usr/bin/landscape-config: error: exit status 1.
2026-03-26T15:07:21.247730+00:00 HP-VICTUS systemd[1]: apport-autoreport.path - Process error reports when automatic rep
orting is enabled (file watch) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/auto
report).
2026-03-26T15:07:21.247741+00:00 HP-VICTUS systemd[1]: apport-autoreport.timer - Process error reports when automatic re
porting is enabled (timer based) was skipped because of an unmet condition check (ConditionPathExists=/var/lib/apport/au
toreport).
2026-03-26T15:07:21.250311+00:00 HP-VICTUS kernel: RAS: Correctable Errors collector initialized.
2026-03-26T15:08:05.472694+00:00 HP-VICTUS kernel: WSL (244) ERROR: CheckConnection: getaddrinfo() failed: -5
```

This script reads a log file and counts the number of lines containing a specific keyword (default: "error"). It uses a while loop to read the file line by line and increments a counter whenever the keyword is found.

Additionally, the script displays the last five matching lines using grep and tail. This script demonstrates file handling, loops, conditional statements, and command-line arguments.

## Script 5: Open Source Manifesto Generator

```
hp@HP-VICTUS:~/oss-project$ nano script5.sh
hp@HP-VICTUS:~/oss-project$ chmod +x script5.sh
./script5.sh
```

```
===== Open Source Manifesto Generator =====

1. Name one open-source tool you use: Git
2. What does freedom mean to you (one word)? freedom
3. What will you build and share? AI tools
```

```
Manifesto saved to manifesto_hp.txt
---------
On 26 March 2026, I declare my belief in open source.
I use Git and believe in freedom.
I will contribute by building AI tools and sharing it freely.
Knowledge should be open, accessible, and collaborative.
hp@HP-VICTUS:~/oss-project$
```

This script interacts with the user by asking questions about their views on open source software. Based on the user's input, it generates a personalized manifesto and saves it to a text file.

The script uses the read command for user input and file redirection (>, >>) to store the output. It demonstrates string manipulation, user interaction, and file handling in shell scripting.

## 7 . Conclusion

In conclusion, this project provided a deep understanding of open-source software and its importance in modern computing. By analyzing Git, we explored its origin, license, and role in the software ecosystem.

The practical implementation using shell scripts helped in understanding Linux commands and automation. This project highlighted how open-source tools promote collaboration, transparency, and innovation.

Overall, the project enhanced both theoretical knowledge and practical skills related to open-source systems.